Relatório de Evolução do Projeto Integrado

Grupo Enrolados

Guilherme Barioni
Iury Xavier da Silva Mangueira
Lilian Mercedes Paye Conde
Marcus Vinícius Duque
Murilo de Souza Vieira

1 – Avanços realizados

01- Aplicativo Android: O que foi feito: evoluíram as telas e o fluxo de treino livre e guiado, a conexão Bluetooth com a raquete e o sistema de desafios personalizados com pontuação. O sistema conta com uma experiência gamificada onde o usuário cria, publica e participa de desafios, com a oportunidade de ganhar xp, acumular combos, gravar o seu high score e comparar com a pontuação de outros usuários. Todos os dados agora são armazenados no celular e enviados a um servidor remoto. Resultado: Uma experiencia que engaja o usuário e promove a competitividade e busca criar uma sensação de comunidade entre os usuários da plataforma.

02- Visão computacional: Utilizando Python, criamos um modelo de detecção da bola de tênis, acessível através de uma interface Streamlit, conectado a uma câmera ligada ao computador, ele é capaz de acompanhar a trajetória da bola e calcular a sua velocidade. Os dados coletados durante o treino são enviados ao celular do usuário e permitem a aplicação de treinos com objetivos de acertar certos pontos na parede. Pensando na acessibilidade de uso do nosso aplicativo, decidimos que o sistema de visão computacional é opcional no uso do SmartAce, sendo possível treinar somente com o celular e o dispositivo IoT. Resultado: Criamos uma experiencia mais interligada que consegue entregar métricas mais precisas com o auxilio das câmeras.

03-Firmware da Raquete: Buscando melhorar a precisão da detecção dos golpes, reescrevemos o firmware do ESP32 com o sensor MPU-6050, que agora extrai 30 características de cada movimento (antes eram 11), combinando acelerômetro e giroscópio para estimar a orientação, as fases e a trajetória do golpe. O sistema passou de 3 para 13 classes: saque (plano, slice e kick), smash, forehand e backhand (drive, slice e voleio), amortida e lob. O firmware também permite treinar o modelo no próprio dispositivo, com histórico editável de golpes. Resultado: Um dispositivo que reconhece uma variedade de golpes muito mais próxima de um treino real e envia ao aplicativo o tipo de golpe, a velocidade estimada e a presença de impacto, alimentando os desafios e as métricas de desempenho.

2 – Alterações no planejamento

1-Simplificação do conjunto físico: A proposta inicial previa piezoelétrico. O desenvolvimento priorizou a detecção do movimento com o acelerômetro e o giroscópio do MPU-6050, retirando o piezoelétrico do escopo atual. A decisão reduz componentes, fios e custo; apesar da detecção do movimento ser precisa, o limite é que movimento da raquete não comprova, por si só, o contato real da bola.
2-Independência do sistema de câmeras: Inicialmente previámos que o aplicativo estaria fortemente acoplado a o sistema de detecção com câmeras, mas durante o desenvolvimento notamos

que a instalação das câmeras e a necessidade de um computador capaz de rodar um modelo de detecção, são custosos demais para uma aplicação real no mercado. O sistema atualmente conta com uma aba especifica para exercícios com o sistema de câmeras, mas é perfeitamente possível utilizar o aplicativo somente com um celular e o dispositivo IoT.

3 – Evidência de desenvolvimento

```
if (mode == MODE_CLASSIFY) {
  if (!mdl.ok || mdlDirty) { if (!computeModel(false)) { out("# sem modelo\n"); return; } }
  float dk[NCLS]; classDist(f, -1, dk);
  Pred p = decide(dk);
  swingNum++;
  if (p.c1 < 0) { out("# sem classes ativas\n"); return; }
  if (p.reject) {
    sessUnknown++;
    out("#%-3lu  ???  nao identificado (perto de %s, d=%.2f lim=%.2f) w=%.0f\n", swingNum,
        CLSAB[p.c1], p.d1, mdl.rej[p.c1] * rejectK, f[F_GPK]);
    drillFeed(p.c1, true, si.vHeadKmh);
    return;
  }
  int c = p.c1;
  sessCount[c]++; sessSum[c] += si.vHeadKmh; if (si.vHeadKmh > sessMax[c]) sessMax[c] = si.vHeadKmh;
  out("#%-3lu  >> %-12s %3.0f%%  ~%.0fkm/h a=%.1fg %s\n", swingNum, CLS[c], p.conf * 100.0f,
      si.vHeadKmh, f[F_APK], si.impact ? "" : "(sem impacto?)");
  if (p.ambig && p.c2 >= 0)
    out("        ambiguo: %s %.2f x %s %.2f\n", CLSAB[c], p.d1, CLSAB[p.c2], p.d2);
  if (verbose)
    out("        elev=%+.0f dYaw=%+.0f back=%.0fms fwd=%.0fms imp=%+.0fms\n",
        f[F_PATHELEV], f[F_DYAW], f[F_TBACK], f[F_TFWD], f[F_TIMP]);
  drillFeed(c, false, si.vHeadKmh);
}
```

Trecho do firmware presente no esp32. O trecho verifica se existe modelo válido, calcula a classe mais próxima e permite rejeitar o movimento como não identificado quando a distância excede o limite configurado.
O firmware não força uma classificação em todos os casos: exige um modelo treinado e compara as características do movimento com as classes previstas. Isso documenta uma implementação concreta sobre a classificação dos golpes.

4 – Próxima etapa e dificuldades

Principal dificuldade atual: Obter a classificação consistente de golpes em diferentes configurações e intensidades. Os limiares de disparo e o limite de rejeição podem gerar falsos positivos ou movimentos não reconhecidos; a posição do sensor na raquete e a variação entre jogadores alteram os sinais medidos. A velocidade da bola na câmera também não pode ser interpretada como velocidade física real sem calibração adequada.

Próxima ação concreta: Montagem do protótipo na raquete. Até o momento, todos os testes foram realizados com o esp32 acoplado a uma protoboard, para testarmos a eficacia real da proposta, o sistema deve funcionar acoplado a uma raquete e testado em um treino real. Já compramos a raquete e estamos planejando como montar o protótipo final sem incomodar o usuário e desbalancear o peso da raquete.